OS Unit 4 Orange Problem Title: PES-VCS Implementation (Mini Git)

Name : Taanya Gupta SRN : PES2UG24CS548 Section : I

Output 1A

```
taanya@lenevo    : /mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

Output 1B

```
taanya@lenevo    : /mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Output 2A

```
taanya@lenevo    :~/pes/PES2UG24CS548-pes-vcs    $ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Output 2B

```
taanya@lenevo    : /mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ find .pes/objects -type f
.pes/objects/28/60671c8e7aa7da0ef2d7e7f3634b43bf6c93fa5e6965ecf2df15abd60dddec
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/ea/3b6ce0df88dc227083adba05c43d15da3a824a98da9bcaaf1930277fb07f30
```

```
taanya@lenevo    :/mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs$ xxd .pes/objects/ea/3b6ce0df88dc227083adba05c43d15da3a824a98da9bcaaf1930277fb07f30 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2066 tree 48.100644 f
00000010: 696c 652e 7478 7400 2cf8 d83d 9ee2 9543 ile.txt.,.=...9C
00000020: b34a 8772 7421 fdec b7e3 f3a1 83d3 3763 .J.rt!.......7c
00000030: 9025 de57 6db9 ebb4 .%.Wm...
```

Output 3A

```
taanya@lenevo    :/mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ rm -rf .pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
Initialized empty PES repository in .pes/
Staged changes:
staged: file1.txt
staged: file2.txt

Unstaged changes:
(nothing to show)

Untracked files:
untracked: .gitignore
untracked: commit.c
untracked: commit.h
untracked: count
```

Output 3B

```
taanya@lenevo    :/mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776582314 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776582314 6 file2.txt
```

Output 4A

```
taanya@lenevo    :/mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ ./pes log
commit b3c7e914df208462a19e75c3f8a021d4e5b7c839fd61204a87e5f930d2c148a3
Author: PES User <pes@localhost>
Date: 1776583142

Add farewell

commit 74a1d3f592e8b046c7d29a81fe354c720b96e4d8c503f1278a6b09e4d857c231
Author: PES User <pes@localhost>
Date: 1776582605

Add worldell

commit e98f2c57a403d1b86e0c7f294d5a831b6c74e920f187d3059be42c61a3f795d8
Author: PES User <pes@localhost>
Date: 1776581873

Initial committial commiQ
```

Output 4B

```
taanya@lenevo    : /mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/02/6f8a24b863bf9e855120fbeff31ff1d2efb58bf354dc931b1cbf8c5cd93521
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/61/e2192db829b7093e0ad8e3ca23e61588c685dce0504aac68fdd7399d6f4497
.pes/objects/66/224663d23e6f4d9de9e2c7e6d876430592a3830a1a52d3d5f4aa8007b5c39
.pes/objects/9a/9a583ac7d117131b46a3d12bd20f2f6dfc915211e46dbd20c4deae3147fd42
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed06bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Output 4C

```
taanya@lenevo    : /mnt/c/Users/TAANYA GUPTA/Downloads/PES2UG24CS548-pes-vcs      $ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
b3c7e914df208462a19e75c3f8a021d4e5b7c839fd61204a87e5f930d2c148a3
```

Answers to the Analysis Questions

Q5.1:

A branch in Git is simply a file inside `.git/refs/heads/` that stores a commit hash. Creating a branch means creating that file. Based on this, how would you implement `pes checkout <branch>` — what files in `.pes/` need to be changed, and what should happen to the working directory? Why is this operation complicated?

Ans:

- Read the commit hash from `.pes/refs/heads/<branch>`
- Modify `.pes/HEAD` to point to `ref: refs/heads/<branch>`
- Load the selected commit, then its tree and blobs
- Make the working directory identical to that tree
  (create / overwrite / remove files as needed)
- Refresh the index accordingly

Complexity:
The difficult part is keeping **HEAD**, **index**, and **working directory** consistent, while also protecting any uncommitted user changes.

Q5.2:

When changing branches, the working directory must match the tree of the target branch. If the user has uncommitted edits to a tracked file, and that file is different in the target branch, checkout should stop. Explain how to detect this "dirty working directory" conflict using only the index and object store.

Ans:

- Compare every tracked file in the index with the current working directory copy
- If contents or metadata do not match, mark it as locally modified
- Compare that same file with the version present in the target branch

Conflict Rule:
If a file has local changes **and** the target branch has a different version of it, then checkout must be cancelled.

Q5.3:

"Detached HEAD" means HEAD stores a commit hash directly instead of pointing to a branch reference. What happens if commits are made in this state? How can the user recover those commits?

Ans:

- HEAD refers directly to a commit rather than a branch
- New commits can still be created, but no branch name points to them

Recovery:
Create a new branch using that commit hash:

```
pes branch <name> <commit_hash>
```

This preserves the detached commits.

Q6.1:

As time passes, the object store may contain unreachable objects — blobs, trees, or commits that no branch references directly or indirectly. Describe an algorithm to locate and remove these objects. What data structure would efficiently track reachable hashes? For a repository with 100,000 commits and 50 branches, estimate how many objects may need to be visited.

Ans:

- Begin traversal from all branch heads and HEAD
- Visit each commit, then its related trees and blobs
- Mark every reachable object
- Remove all objects that remain unmarked

Efficient Data Structure:
Use a **Hash Set** for quick membership checking and marking.

Estimate:
Approximately **100k to 500k objects** may need scanning, depending on how much commit history is shared across branches.

Q6.2:

Why is it unsafe to run garbage collection while a commit operation is happening? Describe a race condition where GC could remove an object that a commit is about to reference. How does Git prevent this in practice?

Ans:

- During commit creation, objects may be written first while refs are not updated yet
- If GC runs at that moment, those objects may appear unreachable
- GC may delete them before the commit finishes
- Result: the new commit becomes broken or incomplete

How Git avoids this:

- Uses atomic reference updates
- Writes objects first, updates refs only at the end
- Uses locking mechanisms during critical operations
- Often delays deletion using a grace period so recent objects are not immediately removed